AMM Arbitrage 策略：订单簿流动性处理机制详解

作者: AI Assistant

日期: 2025-10-28

相关代码: `amm_arb.py` , `utils.py` , `order_book.pyx`

🎯 核心问题

问题：在 CEX 的订单簿中，如果没有足够的流动性来完成指定数量的交易，AMM Arb 策略会如何处理？是否会根据实际可用量来决定套利？

答案：会！但不是你想象的那样

📊 完整流程解析

1. 套利提案生成流程

```
# utils.py: create_arb_proposals()
async def create_arb_proposals(
        market_info_1: MarketTradingPairTuple,
        market_info_2: MarketTradingPairTuple,
        market_1_extra_flat_fees: List[TokenAmount],
        market_2_extra_flat_fees: List[TokenAmount],
        order_amount: Decimal
) -> List[ArbProposal]:
```

步骤 1: 获取报价（第 32-35 行）

```
tasks.append([
    market_info_1.market.get_quote_price(market_info_1.trading_pair, is_buy, order_amou
    market_info_1.market.get_order_price(market_info_1.trading_pair, is_buy, order_amou
    market_info_2.market.get_quote_price(market_info_2.trading_pair, not is_buy, order_
    market_info_2.market.get_order_price(market_info_2.trading_pair, not is_buy, order_
])
```

调用两个关键方法：

- `get_quote_price()` - 获取成交加权平均价格（VWAP）
- `get_order_price()` - 获取订单价格

步骤 2: 检查价格有效性（第 44-45 行）

```
if any(p is None for p in (m_1_o_price, m_1_q_price, m_2_o_price, m_2_q_price)):
    continue  # 跳过这个套利方向
```

关键点：如果任何价格为 `None` ，该方向的套利提案会被**直接丢弃**！

2. VWAP 计算机制（订单簿深度检查）

CEX 连接器实现（ `exchange_base.pyx` ）

```
async def get_quote_price(self, trading_pair: str, is_buy: bool, amount: Decimal) -> De
    """
    对于 CEX 连接器，报价是成交量加权平均价格（VWAP）
    """
    return Decimal(str(self.get_vwap_for_volume(trading_pair, is_buy, amount).result_pr
```

订单簿 VWAP 计算（`order_book.pyx` 第 341-371 行）

```
cdef OrderBookQueryResult c_get_vwap_for_volume(self, bint is_buy, double volume):
    cdef:
        double total_cost = 0
        double total_volume = 0
        double result_vwap = NaN

    if is_buy:
        # 买单：遍历卖盘（ask）
        for order_book_row in self.ask_entries():
            total_cost += order_book_row.amount * order_book_row.price
            total_volume += order_book_row.amount

            # 如果累计量达到需求量
            if total_volume >= volume:
                # 计算最后一档的部分成交
                total_cost -= order_book_row.amount * order_book_row.price
                total_volume -= order_book_row.amount
                incremental_amount = volume - total_volume
                total_cost += incremental_amount * order_book_row.price
                total_volume += incremental_amount
                result_vwap = total_cost / total_volume
                break
    else:
        # 卖单：遍历买盘（bid）
        for order_book_row in self.bid_entries():
            # 同样的逻辑...
            ...

    # 返回结果
    return OrderBookQueryResult(NaN, volume, result_vwap, min(total_volume, volume))
```

🔍 关键机制详解

机制 1: 订单簿流动性充足

场景

```
CEX 订单簿（卖盘）:
  价格 100: 50 个
  价格 101: 100 个
  价格 102: 200 个

策略请求：买入 100 个
```

计算过程

步骤	价格档位	取用量	累计成本	累计量
1	100	50	5,000	50
2	101	50	10,050	100 ✅

结果：

- `total_volume = 100` ✅ 达到需求
- `result_vwap = 10,050 / 100 = 100.5`
- `result_price = 100.5` ✅ **返回有效价格**

策略行为：

- ✅ `get_quote_price()` 返回 `100.5`
- ✅ 套利提案创建成功
- ✅ 继续执行套利

机制 2: 订单簿流动性不足

场景

```
CEX 订单簿（卖盘）:
  价格 100: 20 个
  价格 101: 30 个
  价格 102: 0 个

策略请求：买入 100 个
```

计算过程

步骤	价格档位	取用量	累计成本	累计量
1	100	20	2,000	20
2	101	30	5,030	50
3	102	0	5,030	50 ❌

循环结束：遍历完所有价格档位， `total_volume = 50 < 100`

结果：

- `total_volume = 50` ❌ 未达到需求
- `result_vwap = 5,030 / 50 = 100.6`
- **但是** `result_price` 可能是 `NaN` 或无效值

策略行为：

- ❌ `get_quote_price()` 可能返回 `None`
- ❌ 第 44 行检查： `if any(p is None ...)`
- ❌ **套利提案被丢弃**
- ❌ **不会执行套利**

机制 3: 极端情况 - 订单簿为空

场景

```
CEX 订单簿（卖盘）:
   （空）

策略请求：买入 100 个
```

结果

- `total_volume = 0`
- `result_vwap = NaN`
- `result_price = None`

策略行为:

- ❌ 直接跳过该套利方向
- 📝 日志： `"No arbitrage opportunity"`

💡 策略不会自动调整订单量！

❌ 策略不会做的事

```
# 策略 ❌ 不会这样做:
if available_volume < order_amount:
    adjusted_amount = available_volume  # ❌ 不会自动调整
    execute_arb(adjusted_amount)
```

✅ 策略实际做的事

```
# 策略 ✅ 实际逻辑:
result = get_vwap_for_volume(order_amount)
if result.result_price is None:
    skip_this_direction()  # ✅ 直接跳过
```

📐 数学证明

为什么不能部分成交?

假设策略允许部分成交:

场景

```
配置: order_amount = 100
市场 1: 只能买入 50 (流动性不足)
市场 2: 可以卖出 100
```

问题 1: 资金不匹配

```
市场 1 买入: 50 × 100 = 5,000 USDT
市场 2 卖出: 100 × 99 = 9,900 USDT

结果: 持有 50 个现货 + 4,900 USDT
风险: 价格下跌 → 亏损
```

问题 2: 利润计算失效

```
原计划利润 = (99 - 100) × 100 = -100 (亏损, 不会执行)
实际情况 = (99 - 100) × 50 = -50 (仍然亏损)

但如果:
原计划利润 = (102 - 100) × 100 = 200
实际情况 = (102 - 100) × 50 = 100

利润减半! 不符合 min_profitability 预期
```

🛠️ 实战影响

场景 1: 高流动性交易对 (如 ETH-USDT)

```
order_amount: 1.0  # 1 ETH
```

订单簿深度:

```
Gate.io ETH-USDT 卖盘:
  3799.5: 5.2 ETH
  3799.6: 10.8 ETH
  3799.7: 15.3 ETH
  ...
```

结果:

- ✅ 流动性充足（>1 ETH）
- ✅ VWAP 计算成功
- ✅ 套利正常执行

场景 2: 低流动性交易对（如 PING-USDT）

```
order_amount: 500  # 500 PING
```

订单簿深度:

```
MEXC PING-USDT 卖盘:
  0.0308: 50 PING
  0.0310: 100 PING
  0.0315: 80 PING
  总计: 230 PING < 500 PING ❌
```

结果:

- ❌ 流动性不足（230 < 500）
- ❌ `get_quote_price()` 返回 `None`
- ❌ 套利提案被丢弃
- 📝 日志: `"No arbitrage opportunity"`

解决方案:

```
# 降低订单量以匹配流动性
order_amount: 200  # 200 PING < 230 ✅
```

场景 3: 单边流动性不足

```
市场 1 (DEX): 可以买入 500 PING
市场 2 (CEX): 只能卖出 230 PING ❌
```

结果:

- 方向 1（M1买 M2卖）: ❌ CEX 流动性不足，提案丢弃
- 方向 2（M1卖 M2买）: ✅ 可能成功（如果反向流动性充足）

关键: 策略会**双向检查**，只执行流动性充足的方向

📊 监控和诊断

如何检查流动性问题?

方法 1: 查看日志

```
# 寻找流动性不足的线索
grep "No arbitrage opportunity" logs_conf_amm_arb_PING.log
```

如果频繁出现:

- 可能是流动性不足
- 或价格差异不够

方法 2: 使用 Hummingbot 命令

```
# 在 Hummingbot 中检查订单簿深度
>>> ticker --exchange gate_io --trading_pair PING-USDT

# 或使用 Gateway 测试 DEX 报价
>>> gateway test-swap uniswap/router ethereum-base PING USDC 500 BUY
```

方法 3: 添加自定义日志

修改 `utils.py`：

```
async def create_arb_proposals(...):
    ...
    # 添加调试日志
    for trade_direction, task_group_result in zip(TradeDirection, results_raw):
        m_1_q_price, m_1_o_price, m_2_q_price, m_2_o_price = task_group_result

        # 🔍 调试信息
        if any(p is None for p in (m_1_o_price, m_1_q_price, m_2_o_price, m_2_q_price))
            logger.warning(
                f"流动性不足或价格无效: "
                f"Market 1 - Quote: {m_1_q_price}, Order: {m_1_o_price}, "
                f"Market 2 - Quote: {m_2_q_price}, Order: {m_2_o_price}, "
                f"Direction: {'BUY' if is_buy else 'SELL'}, "
                f"Amount: {order_amount}"
            )
            continue
```

🎯 最佳实践

1. 订单量设置

评估流动性

```
# 推荐：订单量 = 订单簿深度的 20-30%
available_depth = sum_of_top_10_levels  # 前 10 档总量
safe_order_amount = available_depth * 0.25  # 25%
```

实例

```
# PING-USDT 在 MEXC
# 前 10 档总量: 800 PING
order_amount: 200  # 800 × 25% = 200 ✅
```

2. 分级订单策略

```
# 配置多个策略实例，不同订单量
# Instance 1: 小额高频
order_amount: 50
min_profitability: 0.01  # 1%

# Instance 2: 中额中频
order_amount: 200
min_profitability: 0.015  # 1.5%

# Instance 3: 大额低频
order_amount: 500
min_profitability: 0.02  # 2%
```

3. 动态流动性监控

```
# 伪代码：未来优化方向
class DynamicOrderAmount:
    def adjust_order_amount(self):
        current_depth = self.get_order_book_depth()

        if current_depth > 1000:
            return 500  # 大额
        elif current_depth > 500:
            return 200  # 中额
        else:
            return 50   # 小额
```

🚨 常见错误和解决方案

错误 1: 订单量设置过大

症状：

```
日志一直显示：No arbitrage opportunity
但价格差异明显存在
```

诊断:

```
# 检查订单簿深度
>>> ticker --exchange gate_io --trading_pair PING-USDT
# 比较 order_amount 和 实际深度
```

解决:

```
# 降低订单量
order_amount: 50  # 从 500 降到 50
```

错误 2: 误以为策略会自动调整

错误理解:

> "策略会根据实际可用量自动调整订单大小"

正确理解:

> "策略会检查流动性，**不足则跳过**，不会调整"

正确做法:

```
# 手动设置合适的订单量
order_amount: 200  # 根据订单簿深度手动调整
```

错误 3: 单边市场流动性不足

症状:

```
日志显示:
- 方向 1 (M1买 M2卖): No opportunity
- 方向 2 (M1卖 M2买): 执行成功
```

原因:

```
Market 2 的卖盘流动性不足
但买盘流动性充足
```

解决：

- ✅ 接受单向套利
- ⚠️ 注意仓位平衡
- 💡 考虑调整 `order_amount`

📝 总结

核心机制

问题	答案
策略会检查流动性吗?	✅ 是，通过 VWAP 计算
流动性不足时会调整订单量吗?	❌ 否，会直接跳过
会尝试部分成交吗?	❌ 否，必须全量成交
如何知道是流动性问题?	📝 查看日志 "No arbitrage opportunity"
解决方案?	💡 手动降低 `order_amount`

设计理由

为什么策略不自动调整订单量?

1. **资金安全**：避免单边持仓
2. **利润一致性**：确保预期利润
3. **风险控制**：全量成交或不执行
4. **简单性**：逻辑清晰，易于调试

最后建议

```yaml
# ✅ 推荐配置
order_amount: <订单簿深度的 20-30%>
min_profitability: 0.02  # 2%，覆盖流动性风险

# ❌ 避免配置
order_amount: <超过订单簿深度>
min_profitability: 0.001  # 0.1%，利润太低
```

🔗 相关文档

- AMM Arbitrage 策略深度解析
- CEX-DEX 无转账套利框架
- 订单簿数据结构

更新日期: 2025-10-28
代码版本: Hummingbot v1.x